AI LAB WEEK 5

Implementation of Simulated Annealing to Solve 8-Queens problem

```
import random
import math

N = 8

# Calculate the number of attacking queen pairs
def heuristic(state):
    h = 0

    for i in range(N):
        for j in range(i + 1, N):
            # Same row or same diagonal
            if state[i] == state[j] or abs(state[i] - state[j]) == abs(i - j):
                h += 1

    return h


# Display the chessboard
def print_board(state):
    for row in range(N):
        for col in range(N):
            if state[col] == row:
                print("Q", end=" ")
            else:
                print(".", end=" ")
```

```
        print()


# Generate one random neighbor
def get_neighbor(state):
    neighbor = state.copy()

    col = random.randint(0, N - 1)
    new_row = random.randint(0, N - 1)

    while new_row == neighbor[col]:
        new_row = random.randint(0, N - 1)

    neighbor[col] = new_row
    return neighbor


# Simulated Annealing algorithm
def simulated_annealing():
    # Initial random state
    current = [random.randint(0, N - 1) for _ in range(N)]
    best = current.copy()

    T = 100.0
    cooling_rate = 0.99
    min_temperature = 0.001

    print("Initial state:", current)
    print("Initial cost:", heuristic(current))

    while T > min_temperature:
```

```
    current_cost = heuristic(current)

    # Goal test
    if current_cost == 0:
        break

    # Generate a random neighbor
    neighbor = get_neighbor(current)
    neighbor_cost = heuristic(neighbor)

    # Calculate change in cost
    delta_E = neighbor_cost - current_cost

    # Accept a better state or probabilistically accept a worse one
    if delta_E < 0 or random.random() < math.exp(-delta_E / T):
        current = neighbor

    # Track the best state found
    if heuristic(current) < heuristic(best):
        best = current.copy()

    # Cool down
    T *= cooling_rate

# Check the final current state too
if heuristic(current) < heuristic(best):
    best = current.copy()

print("\nBest state:", best)
print("Best cost:", heuristic(best))
```

```
    if heuristic(best) == 0:
        print("Solution found!")
    else:
        print("No solution found. Best state is displayed.")

    print("\nFinal chessboard:")
    print_board(best)
```

simulated_annealing()

